MNIST Project Plan: Augmentation and Feature Reduction Layers

Planning document for the first stages of the pipeline.

1. Pipeline overview

This document describes the first two stages of the project: the **augmentation layer** and the **feature reduction layer**. Both feed the supervised classifiers (Part A) and the clustering experiments (Part B). The data is the standard MNIST split provided as CSV files: 60,000 training rows and 10,000 test rows.

The pipeline is built around two parallel paths, **raw** and **contour/edge**, and three ways of turning each image into features. Every classifier and clustering method can then be compared on the same six feature sets.

Assumption. Each CSV row is one image: the digit label in the first column followed by 784 pixel values (0–255). If the files are laid out differently, only the loading step in `src/data_loading.py` changes.

Recommendation. I recommend running the regular augmentations first and extracting contours from the augmented raw images, rather than extracting contours first and augmenting afterwards. Rotating or shearing a 1 px line forces interpolation, which blurs it and can break it into fragments. Extracting the contour last keeps every outline exactly 1 px wide and binary. It also means both paths see identical geometric transformations, which keeps the raw-versus-contour comparison fair. If you prefer your original order, use nearest-neighbour interpolation and accept some broken lines.

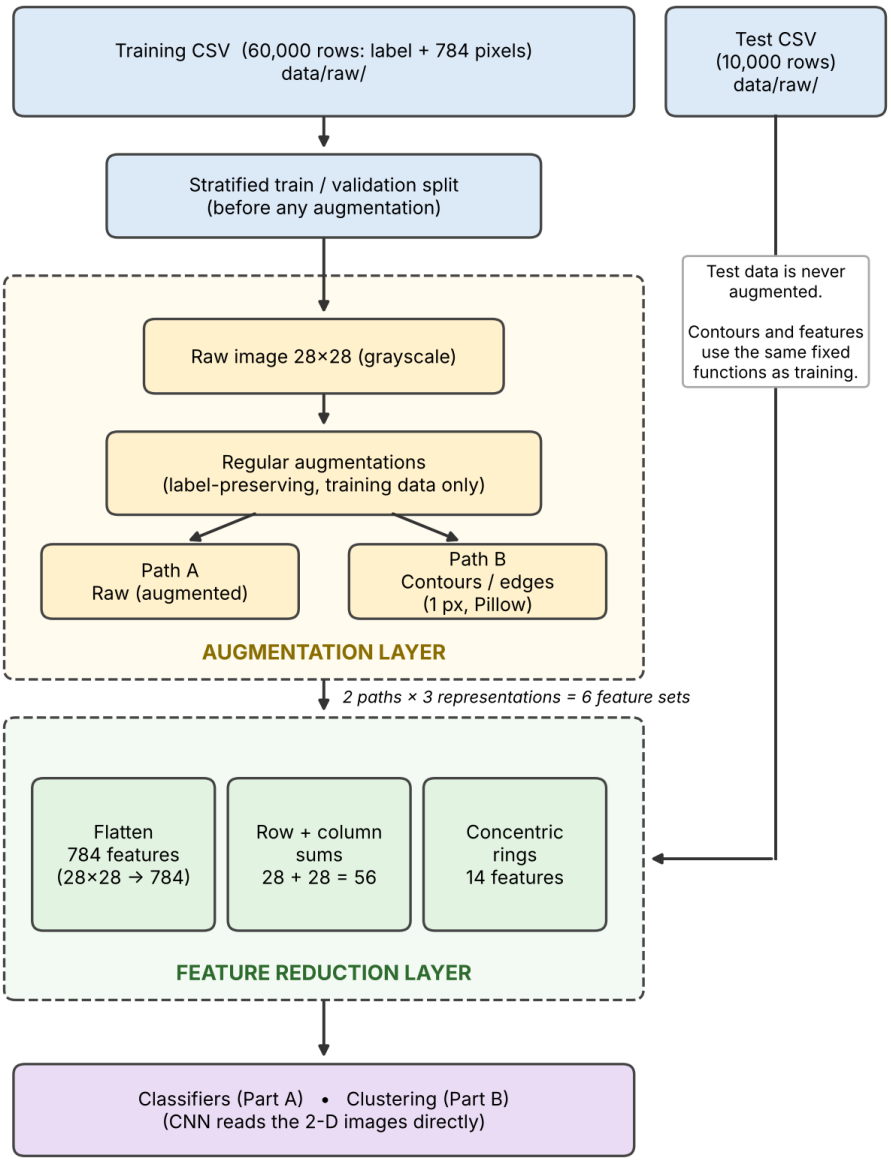


Figure 1. *Planned pipeline. The test set skips augmentation and passes through the same fixed contour and feature functions as the training data.*

2. Project folder structure

The raw CSV files stay untouched in `data/raw/`. Everything derived from them is regenerated by scripts in `src/`, so any result can be reproduced from the raw data.

```
mnist-project/
├── data/
│   ├── raw/                mnist_train.csv, mnist_test.csv  (never modified)
│   ├── contour/            1 px contour images of the clean train/val/test sets
│   ├── augmented/          augmented training copies (raw/ and contour/)
│   └── features/           saved feature matrices: 784, 56 and 14 per path
├── src/
│   ├── data_loading.py     read CSV, stratified train/val split, save .npy
│   ├── augmentation.py     regular augmentations (training data only)
│   ├── contours.py         Pillow contour extraction
│   ├── features.py         flatten, row+column sums, concentric rings
│   ├── classification/     Part A experiments
│   ├── clustering/         Part B experiments
```

```
│   └── utils.py              seeds, plotting, metric helpers
├── results/                  figures, metric tables, saved models
├── docs/                     this plan, report drafts, exhibition material
├── requirements.txt
└── README.md                 how to reproduce the main results
```

Two small choices here. Derived data is stored as `.npy` or `.npz` rather than CSV, because loading 60,000 × 784 values from CSV on every run is slow. The raw files stay as CSV exactly as supplied. The `results/` folder is added so generated figures and tables stay separate from the written documents in `docs/`.

3. Augmentation layer

3.1 Data split before augmentation

The 60,000 training images are first split into a stratified training set and a validation set (for example 54,000 / 6,000). **Augmentation is applied to the training portion only.** The validation and test sets are never augmented. Augmenting before splitting would put near-copies of the same digit on both sides of the split and inflate validation scores, which the project guidelines count as data leakage.

3.2 Path A: raw images

The raw path keeps the grayscale 28×28 image as it is, with pixel values scaled to [0, 1]. Regular augmentations (section 3.4) are applied to it.

3.3 Path B: contours / edges with Pillow

Recommended edge thickness: 1 px. The reason is the stroke width of MNIST digits, which is only about 2–3 pixels. Figure 2 shows the options on two example digits.

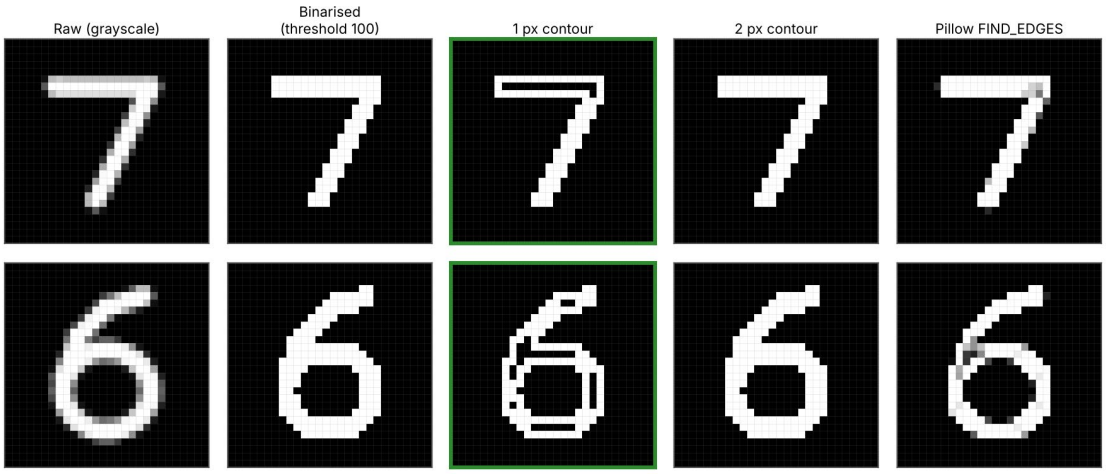


Figure 2. *Edge extraction options on a synthetic 7 and 6 (illustrative drawings, not real MNIST samples). The 1 px contour (green) keeps the digit hollow. At 2 px the outline fills the whole stroke and the result is just a binary copy of the digit.*

Option	What happens	Verdict
1 px contour	Outline of the binarised digit, one pixel wide. Values are exactly 0 or 1.	**Use.** Strokes stay hollow, so path B carries different information from path A.

Option	What happens	Verdict
2 px contour	On a 2–3 px stroke the two sides of the outline meet in the middle and the stroke fills in.	Avoid. It removes the difference between the two paths.
Pillow FIND_EDGES	Gives a gray response that is uneven along the stroke and noisy on faint pixels (last column of Figure 2).	Avoid. Not binary, and sensitive to anti-aliasing.

The contour is built in three steps: binarise at a threshold (about 100 out of 255), erode the mask by one pixel, and subtract the eroded mask from the original. What remains is the one-pixel boundary.

```
from PIL import Image, ImageFilter, ImageChops

def to_contour(img, thr=100):
    mask  = img.point(lambda p: 255 if p > thr else 0)   # binarise
    inner = mask.filter(ImageFilter.MinFilter(3))        # erode by 1 px
    return ImageChops.subtract(mask, inner)              # 1 px outline
```

The output is divided by 255 so that it holds 0/1 values. The threshold is a fixed constant and is not learned from data, so applying it to the test set does not leak information.

3.4 Regular augmentations for digits

Digits are symbols, not natural photographs. Many standard augmentations change a digit's identity or produce shapes no one would write, so each transformation has to pass a simple test: **would a person still read the same digit, and could this image plausibly appear in the test set?**

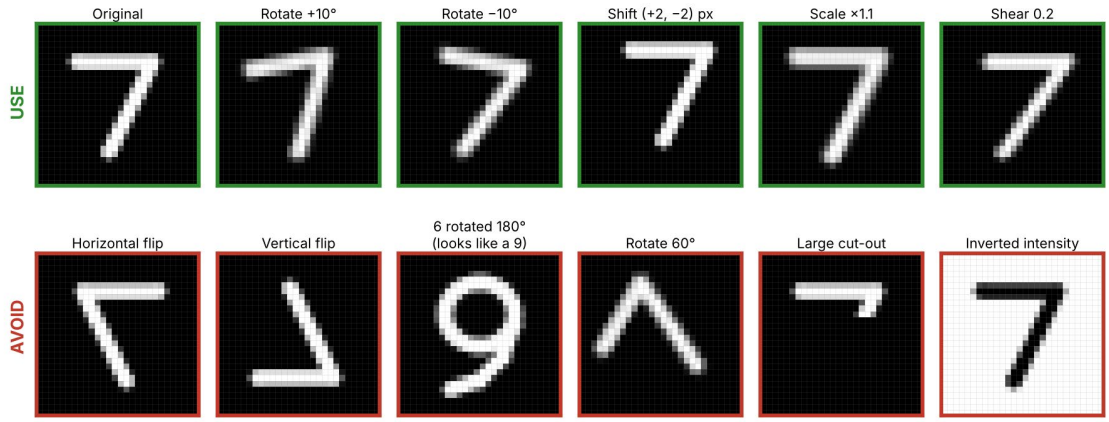


Figure 3. *Top row: label-preserving augmentations. Bottom row: transformations to avoid. Illustrative drawings, not real MNIST samples.*

Augmentations to use

Augmentation	Range	Reasoning
Small rotation	±10° (max ±15°)	People write at slightly different angles. Small tilts stay well inside the same digit.
Translation	±2 px	Position within the frame varies. A larger shift starts to push strokes out of the image.
Scaling	0.9 – 1.1×	Digits differ in size even after MNIST's normalisation.

Augmentation	Range	Reasoning
Shear (slant)	±0.2	Italic-style slant is one of the most common differences between writers.
Elastic distortion (optional)	mild	Simulates hand wobble and has a good track record on MNIST. Keep the strength low so strokes do not tear.
Stroke thickness jitter (optional)	±1 px	Pens differ. Applied to the raw image before contour extraction.

Augmentations to avoid

Augmentation	Problem	Reasoning
Horizontal flip	Mirror-image digits	Digits are not mirror-symmetric. A flipped 2, 3, 4, 5, 7 or 9 is not a valid digit, and would teach the model shapes that never occur in the test set.
Vertical flip	Unnatural shapes	Same issue. No one writes digits upside down.
Rotation of 180° or large angles	Label changes	A 6 rotated by 180° looks like a 9 but keeps the label 6. Even rotations beyond about 20° make 1, 7 and others ambiguous.
Large cut-out / random erasing / aggressive crop	Destroys the evidence	The discriminating part is small: the loop of a 9 versus a 4, or the tail of a 7 versus a 1. Removing it leaves a wrongly labelled image.
Intensity inversion, strong brightness or contrast changes	Not in the test data	MNIST is always light strokes on a dark background. Variations the test set never contains add nothing.

Each training image is kept in its original form and extended with one or two random augmented copies, so the training set is roughly 2–3 times larger. For the CNN, augmentation can instead be applied on the fly each epoch. Random seeds are fixed and stored so the augmented set can be recreated exactly.

4. Feature reduction layer

Both paths, raw and contour, are converted into three feature representations. The CNN is the exception: it reads the 2-D images directly, so it uses the raw and contour images without feature reduction.

4.1 Representation 1: all 784 pixels

The 28×28 image is flattened row by row into one vector of 784 values. This is the baseline that loses no information, and it is the standard input for logistic regression, k-NN, SVM, tree ensembles and the MLP. It is also the input for PCA when we want to see how many dimensions are really needed.

4.2 Representation 2: row and column sums (56 features)

For each of the 28 rows and 28 columns, we add up the pixel values. This gives 28 + 28 = 56 features, a horizontal and a vertical "ink profile" of the digit. (If you meant a running cumulative sum, `np.cumsum`, rather than a total, that is possible too. It carries the same information in a different form, and it matters mainly for distance-based methods.)

Your question about grayscale intensity. Sums work on grayscale images without any special handling. My recommendation:

- **Raw path:** scale pixels to [0, 1] first, then sum. A faint pixel contributes a little and a solid one contributes fully, so the intensity information is kept and the result does not depend on a binarisation threshold. Binarising before summing, which counts ink pixels instead, is a reasonable ablation but it throws information away.
- **Contour path:** the values are already 0/1, so each sum is simply the number of edge pixels in that row or column. No decision is needed.
- **Scaling:** raw sums and contour sums have different ranges, so standardise the features (fitted on the training set only) before distance-based or linear models.

One limitation to expect: a profile records how much ink is in each row and column but not where along it. Different digits can share similar profiles, so I expect this representation to lose some accuracy relative to the full 784. The experiments will show how much.

4.3 Representation 3: concentric rings (14 features)

This matches the spiral in your sketch. Ring 0 is the outermost border of the image, ring 1 is the next layer in, and so on until ring 13, which is the central 2×2 block. A 28×28 image has exactly 14 such rings, hence 14 features.

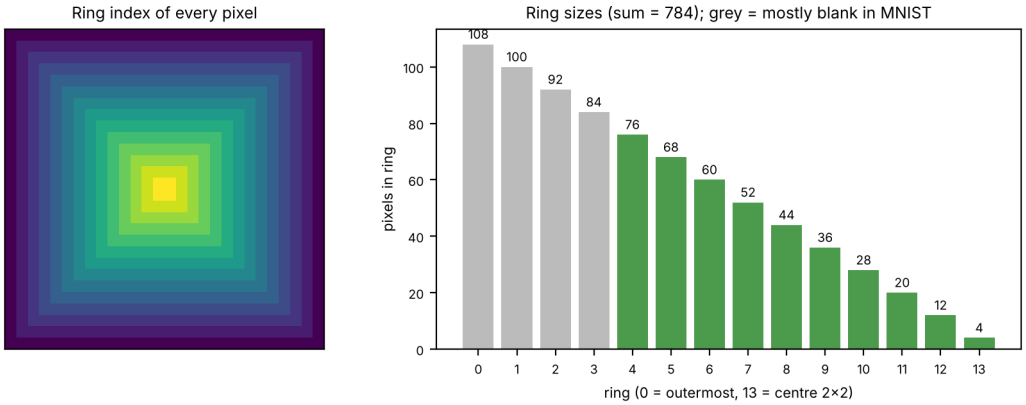


Figure 4. *Concentric ring layout on a 28×28 grid. Rings have very different sizes, from 108 pixels at the border to 4 at the centre.*

My opinion:

- **Use the mean of each ring, not the sum.** The outer ring has 108 pixels and the centre ring only 4, so a plain sum is dominated by ring size. The mean measures ink density and is comparable across rings.
- **The outer rings are almost always blank.** MNIST digits are centred in a 20×20 area, so rings 0–3 carry little information and the effective feature count is nearer ten than fourteen.
- **Expect it to be the weakest representation.** It keeps only radial information, so it cannot tell a 6 from a 9 by shape alone, and it is almost blind to orientation. That is still useful: it gives a deliberately minimal baseline and shows how much accuracy the extra dimensions buy.

If you want a true circular version instead of square rings, the same idea works with Euclidean radius bins. Square rings tile the 28×28 grid exactly, so I suggest starting with those.

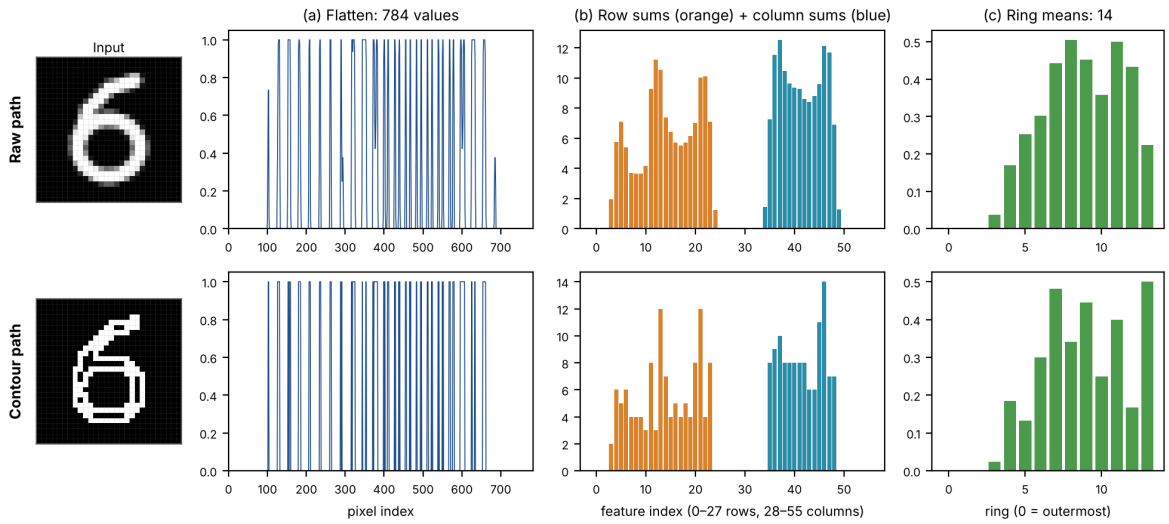


Figure 5. *The three representations for a synthetic 6 on the raw path (top) and the contour path (bottom). (a) 784 flattened values, (b) 28 row sums followed by 28 column sums, (c) 14 ring means. Illustrative drawing, not a real MNIST sample.*

4.4 Resulting feature sets

ID	Path	Representation	Features	Notes
A1	Raw	Flatten	784	Full-information baseline
A2	Raw	Row + column sums (scaled to [0,1])	56	Keeps grayscale intensity
A3	Raw	Ring means	14	Smallest representation
B1	Contour	Flatten	784	Binary 1 px outline
B2	Contour	Row + column sums	56	Counts of edge pixels
B3	Contour	Ring means	14	Counts of edge pixels per ring area
CNN	Raw / contour	2-D 28×28 image	28 × 28	No feature reduction

All feature functions are fixed formulas with no learned parameters, so they are identical for training, validation and test data. Any later step that does learn parameters, such as standardisation or PCA, is fitted on the training set only.

5. Summary of decisions

Question	Recommendation	Why
Edge line thickness	1 px (agree)	MNIST strokes are 2–3 px wide, so 2 px fills the stroke and 1 px keeps it hollow.
Edge method	Binarise, erode by 1 px, subtract (Pillow)	Gives clean 0/1 output. `FIND_EDGES` is noisy and not binary.
Order of operations	Augment first, then extract contours	Keeps 1 px lines crisp and gives both paths identical transformations.

Question	Recommendation	Why
Regular augmentations	Small rotation, shift, scale, shear; optional mild elastic distortion	Label-preserving and plausible in the test set. No flips, 180° rotation or large cut-outs.
Row/column features on grayscale	Scale to [0,1], then sum	Keeps intensity and avoids a threshold. Contour path is already binary.
14-feature circular version	Square concentric rings, mean per ring	Tiles the grid exactly and makes rings comparable. Expect it to be the weakest.
Test set	Never augmented, never used for fitting	Prevents leakage, as required by the project guidelines.